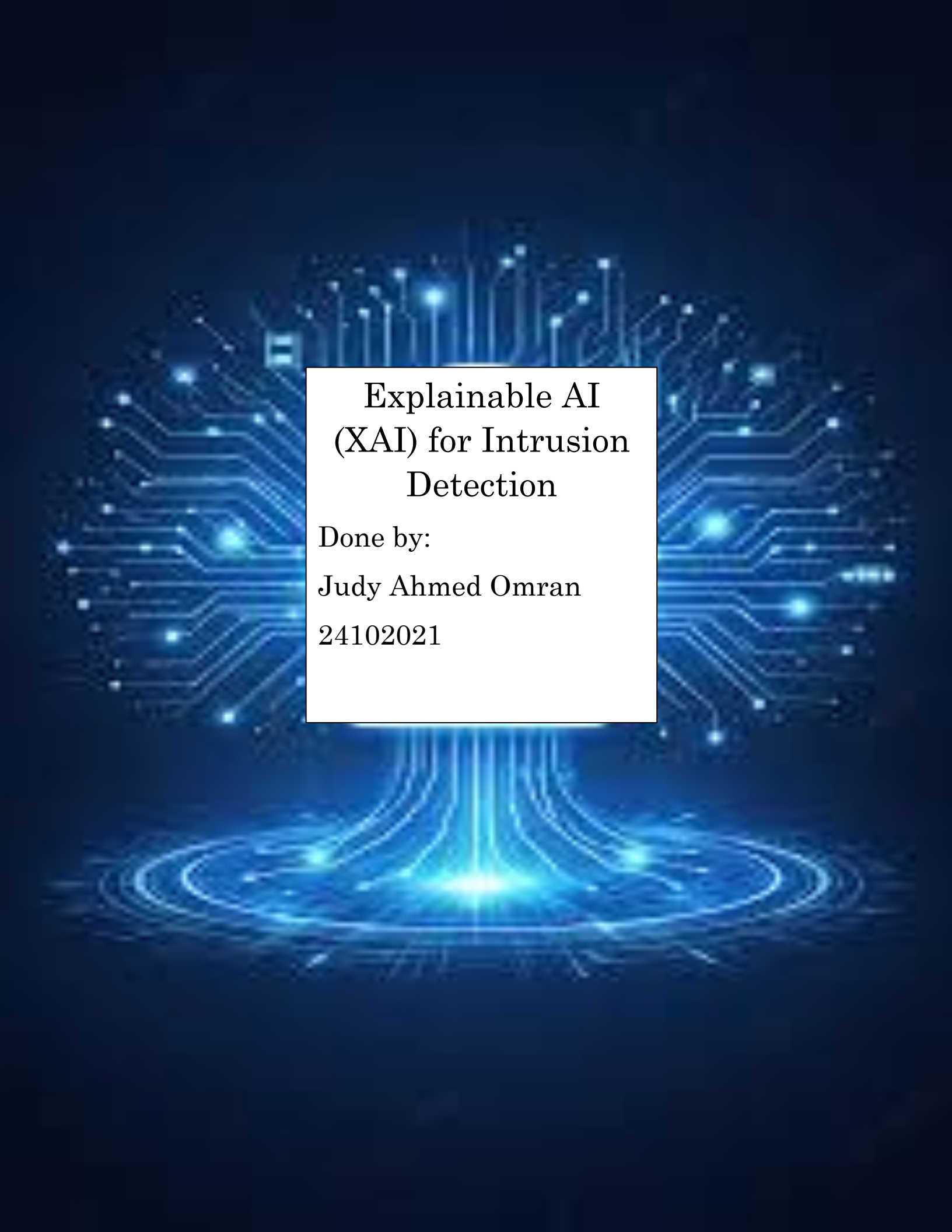




# Explainable AI (XAI) for Intrusion Detection

Done by:

Judy Ahmed Omran

24102021

## Abstract

The project proposes an Explainable Artificial Intelligence (XAI) approach for identifying breaches using machine learning methods. The project uses a set of data called the UNSW-NB15 dataset to train the computer. To make things clearer the project uses two tools called SHAP (SHapley Additive Explanations) and LIME (Local Interpretable Model-Agnostic Explanations) to explain what the computer is thinking. The system provides high performance in classification and clear explanation of decisions. It improves security and usability in cybersecurity systems, allowing analysts to gain greater insight into attack patterns and model behavior.

## Introduction

Intrusion Detection Systems (IDS) are really important for protecting computer networks from cyber threats and unauthorized activities.

Machine learning and deep learning techniques have made these systems better at spotting attacks and abnormal patterns within network traffic.

The UNSW-NB15 dataset has lots of information that helps make these models work better. But even though they are very accurate many machine learning-based IDS models function as “black boxes”, meaning they give predictions without explaining or seeing how they make decisions. This makes it hard to use them in real-life cybersecurity situations.

## Problem Statement

The main problem with machine learning-based IDS models today systems lack transparency, which make Security analysts and practitioners want to understand reasoning behind a model’s prediction so they can trust the results and take the appropriate steps.

This project tries to solve this issue by using Explainable AI techniques like SHAP and LIME on an IDS model that was trained with the UNSW-NB15 dataset. The goal is to make intrusion detection decisions more transparent; this will help make cybersecurity more reliable for real-world use.

## Literature Review

### Paper 1: Explainable AI for Cybersecurity

This paper talks about why we need to add (XAI) to cybersecurity systems, especially Intrusion Detection Systems (IDS).

Traditional machine learning models have really high accuracy but lack of transparency so, they are hard for Security analysts to understand.

The studies looked at XAI methods like SHAP and LIME to help explaining model predictions, which increase the trust and usability for security analysts. the paper mainly does not show how to use real data like the UNSW-NB15 dataset. This leaves a gap that this project tries to fill by using SHAP and LIME on an IDS model that was trained on actual data. The model will help to make things clearer.

### **Paper 2: Intrusion Detection System Using Machine Learning**

The system uses machine learning to figure out if the traffic on the network's normal or if its malicious. In this project I used some models like Random Forest and Support Vector Machines to make the system more accurate. They got high performance in results of detecting attacks. Also, the system worked well especially when they used a bunch of models together.

The problem is the models just give an answer without explaining why they made that choice. Which makes it hard for cybersecurity experts to understand and trust the systems decisions.

### **Paper 3: A Unified Approach to Interpreting Model Predictions**

The study talks about SHAP (SHapley Additive exPlanations) which is a way to figure out which features are important when a model makes a prediction. It does this by using game theory. This means we can understand how the whole dataset works.

LIME (Local Interpretable Model-agnostic Explanations) is a way to explain individual predictions by approximating the model locally using a simpler interpretable model.

The big difference, between SHAP and LIME is that SHAP helps us understand the model. It shows us how all the features work together. LIME is different. It helps us understand why a model made a prediction.

In this project we use SHAP to find the important features that affect how we detect intrusions. We use LIME to explain why the model thought a specific event was an attack or normal traffic. By using both SHAP and LIME we can trust the model more. We get an understanding of how it works.

## **Methodology**

The methodology consists of five main stages, including data collection, preprocessing, model development, evaluation, and explainability integration.

### **1. Data Collection:**

The data we are using for this project is the UNSW-NB15 Dataset. This dataset is really popular when it comes to researching ways to detect intrusions. The UNSW-NB15 Dataset has a mix of network traffic and bad network traffic. It contains:

- 49 features
- Normal and attack traffic
- Various attack types (DoS, Exploits)

This makes the UNSW-NB15 Dataset very good for training machine learning models and for using techniques, like SHAP and LIME to understand what is going on.

A good dataset is important because it helps the model learn real attack patterns, not random noise.

## **2. Data Preprocessing:**

Before we train the model, we need to make sure the dataset is ready.

The dataset is prepared by:

- Dealing with inconsistent values in the dataset.
- Changing features like the protocol type into numbers that the model can understand.
- Choosing the features that can help the model and drop or get rid of the ones that are not useful.
- Splitting the dataset into training and testing sets, We can use most of the dataset like 80% for training and the rest 20% for testing.

## **3. Model Selection:**

Using a machine learning model like Random Forest is to classify things. Random Forest is chosen because it is really good at handling sets of data and it works well in intrusion detection tasks.

Choosing the right model affects how well I can detect things and if the system is stable or not.

## **4. Training and Testing:**

The model is trained using the training dataset and then evaluated using the testing dataset. This is done to see how well the model works.

We look at things like accuracy and precision and recall and F1-score to see how well the model detects attacks. This step verifies if the model is good with data that it has not seen before.

## **5. Using SHAP و LIME:**

To make the model easy to understand we use two techniques:

- SHAP helps in global explanations by finding the most important factors that affect predictions, across all data.
- LIME explains individual predictions by showing which factors led to a decision.

This step turns the model from a “black box” into a transparent system that security analysts can rely on and understand.

# Data Preprocessing Section

## Uploading Files:

```
[5]
✓ 4m
from google.colab import files
uploaded = files.upload()

Choose Files 2 files
unsw_test.csv(text/csv) - 32293018 bytes, last modified: 4/21/2026 - 100% done
unsw_train.csv(text/csv) - 15380800 bytes, last modified: 4/21/2026 - 100% done
Saving unsw_test.csv to unsw_test.csv
Saving unsw_train.csv to unsw_train.csv
```

- files.upload() opens a button in Google Colab so you can upload files from your computer.
- I choose these two files which are datasets:
  - unsw\_train.csv
  - unsw\_test.csv

## Reading and Exploring the Data:

```
[6]
✓ 0s
import pandas as pd

train = pd.read_csv("unsw_train.csv")
test = pd.read_csv("unsw_test.csv")

print(train.shape)
print(test.shape)
train.head()
```

(82332, 45)  
(175341, 45)

|   | id | dur      | proto | service | state | spkts | dpkts | sbytes | dbytes | rate        | ... | ct_dst_sport_ltm | ct_dst_src_ltm | is_ftp_login | ct_ftp_ |
|---|----|----------|-------|---------|-------|-------|-------|--------|--------|-------------|-----|------------------|----------------|--------------|---------|
| 0 | 1  | 0.000011 | udp   | -       | INT   | 2     | 0     | 496    | 0      | 90909.0902  | ... | 1                | 2              | 0            |         |
| 1 | 2  | 0.000008 | udp   | -       | INT   | 2     | 0     | 1762   | 0      | 125000.0003 | ... | 1                | 2              | 0            |         |
| 2 | 3  | 0.000005 | udp   | -       | INT   | 2     | 0     | 1068   | 0      | 200000.0051 | ... | 1                | 3              | 0            |         |
| 3 | 4  | 0.000006 | udp   | -       | INT   | 2     | 0     | 900    | 0      | 166666.6608 | ... | 1                | 3              | 0            |         |
| 4 | 5  | 0.000010 | udp   | -       | INT   | 2     | 0     | 2126   | 0      | 100000.0025 | ... | 1                | 3              | 0            |         |

5 rows × 45 columns

- Imports pandas library for handling datasets.
  - Loads the datasets into DataFrames:
    - train: training data
    - test: testing data
  - **Print train:**
    - train has 82,332 rows, 45 columns
    - test has 175,341 rows, 45 columns
- So, both datasets have the same features (columns)

- **train.head():**
  - id → record number
  - dur → duration of connection
  - proto → protocol (tcp, udp, etc.)
  - service → service type
  - state → connection state

This dataset is network traffic data used for intrusion detection.

## Data Cleaning & Splitting Features:

```
[7]
✓ 0s
train = train.drop(columns=["id", "attack_cat"])
test = test.drop(columns=["id", "attack_cat"])

y_train = train["label"]
X_train = train.drop(columns=["label"])

y_test = test["label"]
X_test = test.drop(columns=["label"])
```

- First drop some categories that is not important
  - id → not useful for ML
  - attack\_cat → attack category (you're not using it now)
- **label column:**
  - 0 → normal traffic
  - 1 → attack

This is what your model will predict.

## Encoding Categorical Data

```
[8]
✓ 0s
from sklearn.preprocessing import OrdinalEncoder

cat_cols = X_train.select_dtypes(include=['object']).columns

encoder = OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1)

X_train[cat_cols] = encoder.fit_transform(X_train[cat_cols])
X_test[cat_cols] = encoder.transform(X_test[cat_cols])
```

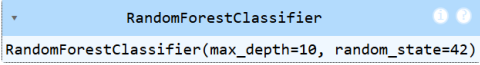
- This imports OrdinalEncoder, which converts text (categorical values) into numbers. Because Machine learning models (like Random Forest) can't handle text directly they need numbers.
- create an encoder object.
- Important parameters:
- fit\_transform does two things:
  1. Learns categories from training data
  2. Converts them into numbers

## Random Forest Model

```
[9] ✓ 11s from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,
    max_depth=10,
    random_state=42
)

model.fit(X_train, y_train)
```



- Imports the **Random Forest algorithm**
- `n_estimators=100`  
The model uses 100 decision trees
- `max_depth=10`  
Each tree can grow up to depth 10 (prevents overfitting)
- `random_state=42`  
Makes results reproducible (same output every run)

```
1 from sklearn.preprocessing import OrdinalEncoder

cat_cols = ['proto', 'state']

for col in cat_cols:
    X_train[col] = X_train[col].astype(str)
    X_test[col] = X_test[col].astype(str)

encoder = OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1)

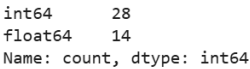
X_train[cat_cols] = encoder.fit_transform(X_train[cat_cols])
X_test[cat_cols] = encoder.transform(X_test[cat_cols])
```

This encoder is used to convert categorical text data (like "TCP", "UDP", etc.) into numeric values so machine learning models can understand them.

### Converting all values to numeric:

```
[11] ✓ 0s X_test = X_test.apply(pd.to_numeric, errors='coerce').fillna(-1)

[12] ✓ 0s print(X_test.dtypes.value_counts())
```



- `pd.to_numeric(errors='coerce') :`
  - Converts everything to numbers
  - If something cannot be converted → becomes NaN
- `.fillna(-1) :`
  - Replaces all NaN values with -1

## Accuracy:

```
[13]
✓ 1s
y_pred = model.predict(X_test)

from sklearn.metrics import accuracy_score, classification_report

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
✓
Accuracy: 0.8999435385905179
      precision    recall  f1-score   support

     0       0.77       0.98       0.86       56000
     1       0.99       0.86       0.92      119341

   accuracy          0.90       0.90       0.90      175341
  macro avg       0.88       0.92       0.89      175341
 weighted avg       0.92       0.90       0.90      175341
```

## Running SHAP:

```
[14]
✓ 9s
!pip install shap

Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.51.0)
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/dist-packages (from shap) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from shap) (1.16.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (from shap) (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from shap) (2.2.2)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.12/dist-packages (from shap) (4.67.3)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-packages (from shap) (26.1)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba in /usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: llvmlite in /usr/local/lib/python3.12/dist-packages (from shap) (0.43.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.2)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from shap) (4.15.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2026.1)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (3.6.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil->=2.8.2->pandas->shap) (1.17.0)
```

## Importing SHAP:

```
[15]
✓ 18s
import shap

X_sample = X_test.sample(1000, random_state=42)

explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_sample)
```

```
[16]
✓ 0s
import numpy as np

X_sample = X_test[:1000]
X_sample = np.array(X_sample)
```

- Selects 1000 random rows from your test dataset.
- Calculates SHAP values for each feature in each sample
- imports NumPy.
- Selects the first 1000 rows from X\_test.( used earlier)
- Converts your data from a pandas DataFrame → NumPy array



## Computing SHAP values and Getting feature names

```
[17]
✓ 8s
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_sample)

[18]
✓ 0s
feature_names = train.drop(columns=['label']).columns.tolist()

[19]
✓ 0s
print(type(shap_values))
print(len(shap_values))
print(type(shap_values[0]))
print(shap_values[0].shape)
print(X_sample.shape)

<class 'numpy.ndarray'>
1000
<class 'numpy.ndarray'>
(42, 2)
(1000, 42)
```

1000 is Number of samples, 42 is Number of features and 2 is Number of classes (Normal / Attack)

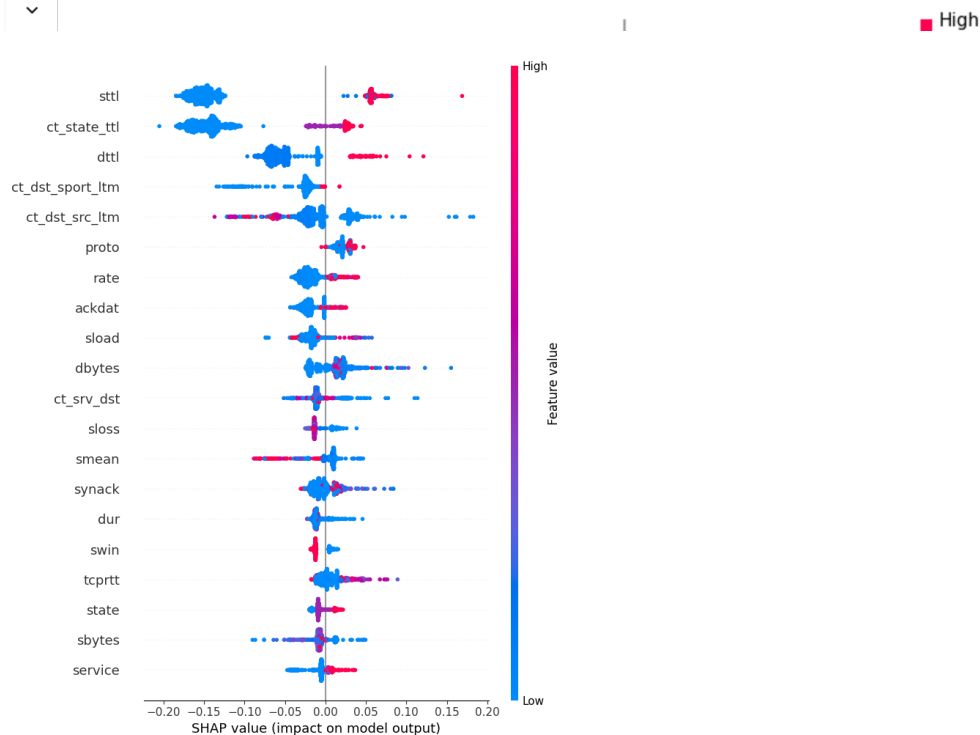
## SHAP-Based Feature Importance Visualization for Model Explanation

```
[21]
✓ 1s
import pandas as pd

X_sample_df = pd.DataFrame(X_sample, columns=feature_names)

sv = shap_values[:, :, 1]

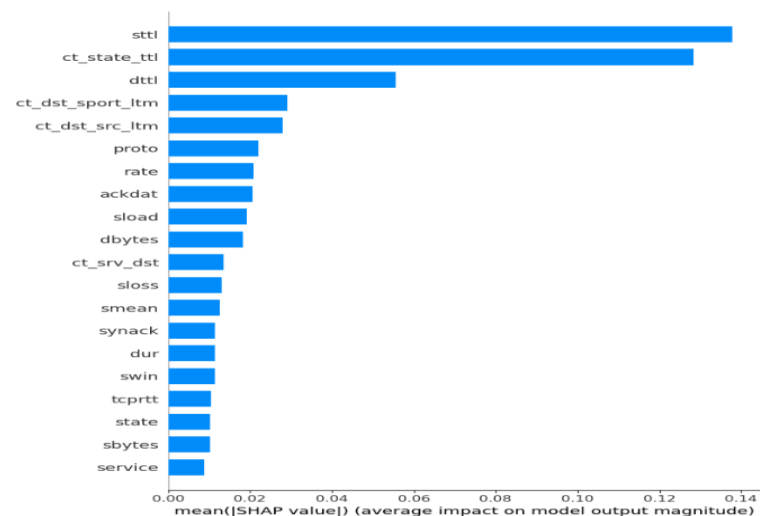
shap.summary_plot(sv, X_sample_df)
```



## SHAP plot (bar chart):

```
[22] shap.summary_plot(sv, X_sample, feature_names=feature_names, plot_type="bar")
```

✓ 0s

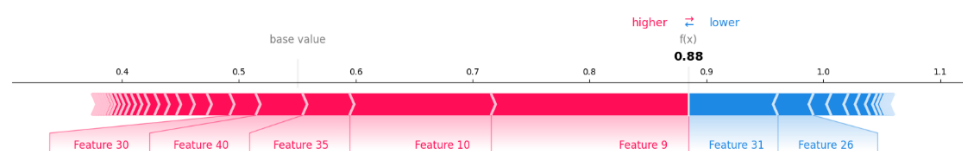


```
[23] import shap
shap.initjs()
```

✓ 0s

```
[24] shap.force_plot(
    explainer.expected_value[1],
    sv[0],
    matplotlib=True
)
```

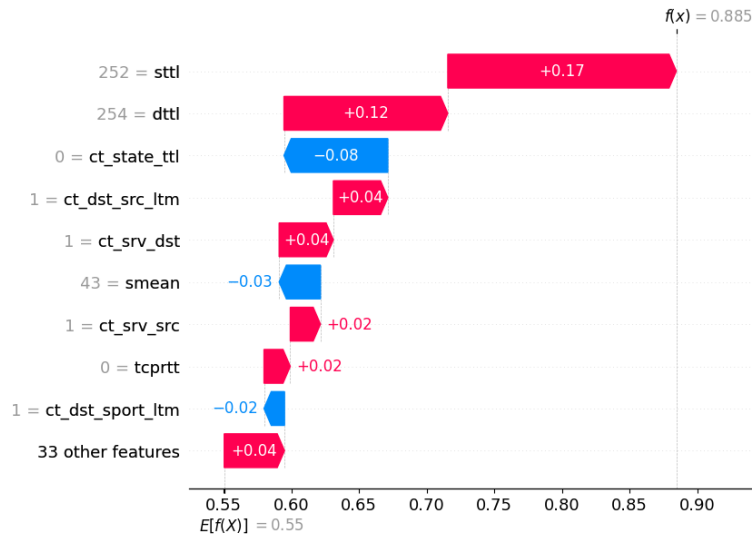
✓ 0s



## The waterfall plot:

```
[25] shap.plots.waterfall(shap.Explanation(values=sv[0],
    base_values=explainer.expected_value[1],
    data=X_sample[0],
    feature_names=feature_names))
```

✓ 0s



## Confusion matrix:

```
[43] ✓ 1s
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)

plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

## ROC curve:

```
[44] ✓ 0s
from sklearn.metrics import roc_curve, auc

fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, label="AUC = %0.2f" % roc_auc)
plt.plot([0,1],[0,1], '--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()
```

## Running LIME:

```
[29]
✓ 7s
❯ pip install lime

Collecting lime
  Downloading lime-0.2.0.1.tar.gz (275 kB)
    275.7/275.7 kB 17.1 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from lime) (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from lime) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from lime) (1.16.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from lime) (4.67.3)
Requirement already satisfied: scikit-learn>=0.18 in /usr/local/lib/python3.12/dist-packages (from lime) (1.6.1)
Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/python3.12/dist-packages (from lime) (0.25.2)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (3.6.1)
Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (11.3.0)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (2.37.3)
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (2026.4.11)
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (26.1)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (0.5)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lime) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lime) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (0.12.1)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.5.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib->lime) (1.17.0)
Building wheels for collected packages: lime
  Building wheel for lime (setup.py) ... done
  Created wheel for lime: filename=lime-0.2.0.1-py3-none-any.whl size=283834 sha256=b0940b86be49537b8b6692c16911712cc46b57736e2b9f80be256c106
  Stored in directory: /root/.cache/pip/wheels/e7/5d/0e/4b4ffff9a47468fed5633211fb3b76d1db43fe806a17fb7486a
Successfully built lime
Installing collected packages: lime
Successfully installed lime-0.2.0.1
```

## LIME explainer setup:

```
[30]
✓ 0s
from lime.lime_tabular import LimeTabularExplainer

[31]
✓ 1s
explainer = LimeTabularExplainer(
    training_data=X_train.values,
    feature_names=X_train.columns,
    class_names=['Normal', 'Attack'],
    mode='classification'
)
```

- Converts training data into a NumPy array
- LIME uses it to understand the distribution of your dataset
- Provides column names (Ensures output is readable)

## Local Explanation of Intrusion Detection Predictions Using LIME

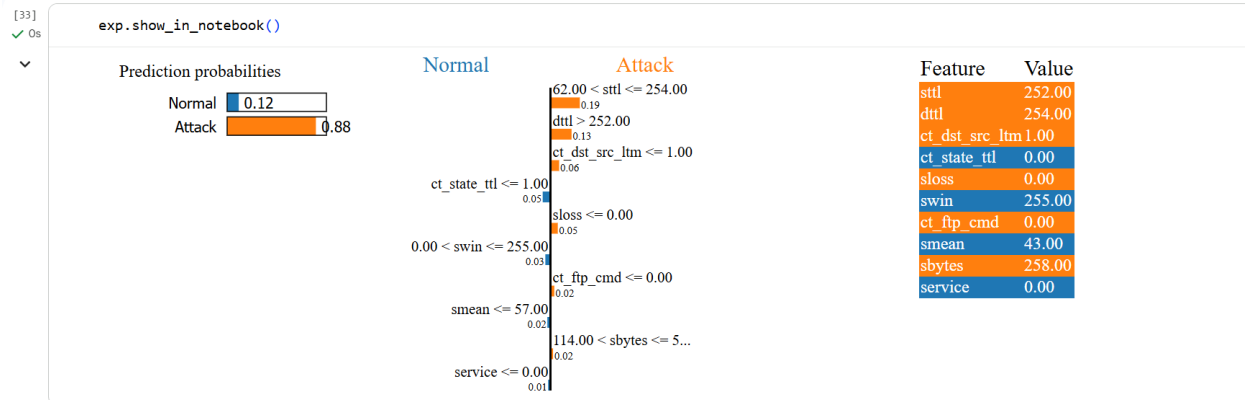
```
[32]
✓ 0s
exp = explainer.explain_instance(
    X_test.iloc[0].values,
    model.predict_proba
)

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but RandomForestC
warnings.warn(
```

This code uses LIME to explain why the model made a prediction for one data sample. It does this by showing which features had the influence on the result. The code also shows that

certain features have an impact on whether a connection is classified as normal or an attack  
these features include things, like: Protocol, Duration and Byte values.

## LIME Local Explanation Visualization for a Single Prediction



## LIME-Based Feature Explanation for a Single Prediction:

[42] ✓ Os

```
exp = explainer.explain_instance(  
    X_test.iloc[0].values,  
    model.predict_proba,  
    num_features=5  
)  
  
exp.as_list()
```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but RandomForestC  
warnings.warn(  
[('62.00 < sttl <= 254.00', 0.18390460125308758),  
( 'ct\_state\_ttl <= 1.00', -0.05484642319562952),  
( 'ct\_dst\_src\_ltm <= 1.00', 0.051229857512090124),  
( 'sloss <= 0.00', 0.04809218928033853),  
( '0.00 < swin <= 255.00', -0.031465670497195126)]

- explain\_instance(...) generates an explanation for a single test sample.
- model.predict\_proba tells LIME to use the model's probability outputs.
- num\_features=5 means only the 5 most important features will be shown.

## Saving and Downloading Machine Learning Model:

[45] ✓ Os

```
import joblib  
  
joblib.dump(model, "ids_model.pkl")  
print("Model saved successfully!")
```

Model saved successfully!

[46] ✓ Os

```
from google.colab import files  
files.download("ids_model.pkl")
```

## Results

### 2. Accuracy

```
y_pred = model.predict(X_test)

from sklearn.metrics import accuracy_score, classification_report

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.8999435385905179
              precision    recall  f1-score   support

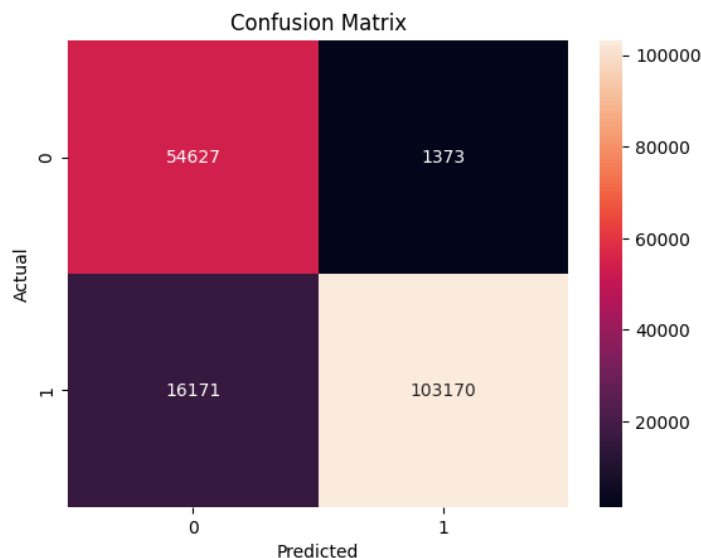
     0:       0.77       0.98       0.86       56000
     1:       0.99       0.86       0.92      119341

 accuracy
macro avg       0.88       0.92       0.89      175341
weighted avg     0.92       0.90       0.90      175341
```

The intrusion detection model we are looking at does a good job of figuring out what is normal network traffic and what is malicious. The accuracy is 90% this means the model is good at telling the difference between malicious and normal traffic just because it is accurate does not mean it is perfect. We need to look at more details to see how well it really works.

The report shows that the model performs differently across the two classes. For class 0 (normal traffic), the model achieved a precision of 0.77 and a very high recall of 0.98, that means it identifies most normal traffic directly but includes some false positives. For class 1 (attack traffic), the model achieved a very high precision of 0.99 and a recall of 0.86, which shows that most predicted attacks are correct, but also some actual attacks may still be undetected.

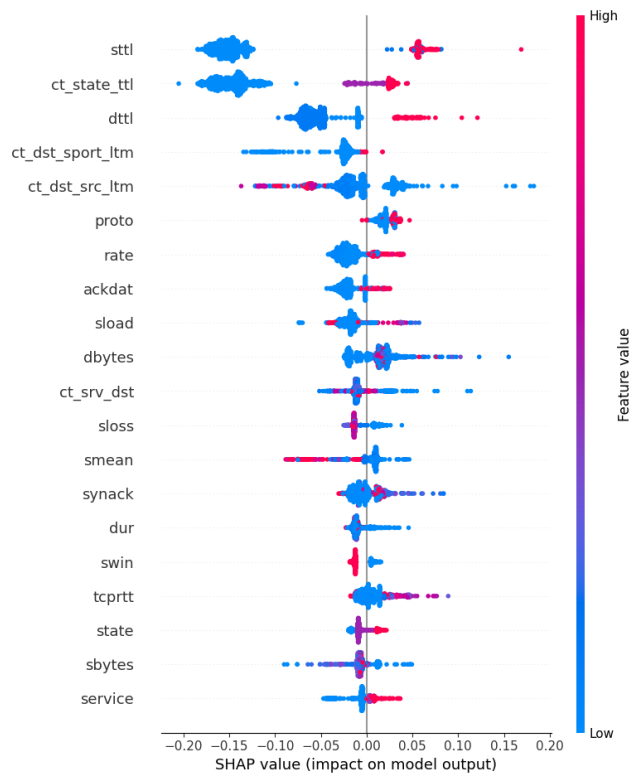
### 3. Confusion Matrix



The confusion matrix helps us see how well our model works by comparing labels with predicted labels. Our model got 54,627 normal network traffic instances right. It also correctly identified 103,170 attack instances. This shows our model is good at detecting problems.

However, we have 1,373 cases where normal traffic was wrongly flagged as an attack. The issue is that 16,171 attack instances were missed and marked as traffic. In cybersecurity missing attacks is a deal because it can put the system at risk. With these issues our model is good at spotting malicious activities.

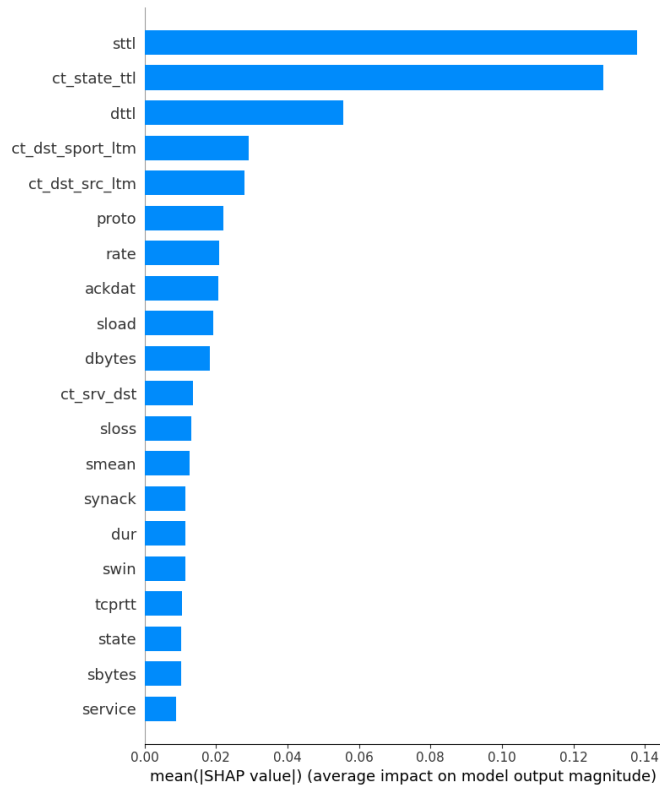
#### 4. SHAP Summary Plot:



This graph shows the features importance and their impact on the model's prediction. For example, in this graph:

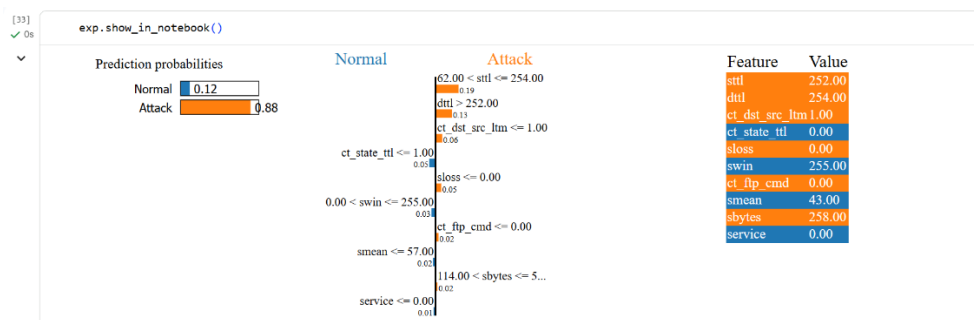
**ct\_state\_ttl** is a top one which means its very important and also it has a wide spread which means it has a strong impact on model decisions

## 5. SHAP Feature Bar Plot:



It shows the average impact of each feature. The first few bars are the longest which means they have the greatest influence on the model.

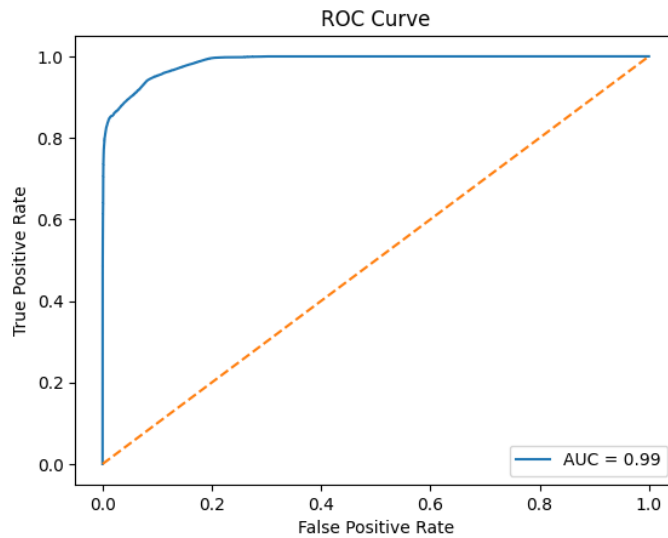
## 6. LIME Explanation:



It explains individual predict at the time. In this graph, each feature either support or goes against the predictions.



## 7. ROC Curve:



The blue curve show the performance of the model, the AUC is 0.99 which means it perform very well and can accurately detect attacks.

## Discussion

The results from this project show that the machine learning system for detecting intrusions works well in figuring out what is going on with network traffic using the UNSW-NB15 dataset. The model did a job overall and was good at finding the right balance between precision and recall and F1-score metrics. This means it is good at detecting malicious activities.

## Conclusion

This project was able to create a system that can detect intrusions in a way that people can understand. It used machine learning and other techniques like SHAP and LIME to make it clear how the system made its decisions. It was trained on the UNSW-NB15 dataset. The system was able to do this because it used Explainable AI, This is really important for cybersecurity because people need to be able to trust the system and understand how it is working.

## Future Work

Future improvements to this project can focus on making the intrusion detection system work better and be easier to understand.

- One way to do this is to use advanced models such as LSTM or Transformer-based architectures. These models could help improve detection accuracy for network patterns.
- Implement real-time intrusion detection systems, this would allow the system to analyze network traffic instead of just static datasets.

Overall, these enhancements would make the system more suitable, for real-world cybersecurity environments

## References

1. IEEE Paper on XAI (2022)
2. UNSW-NB15 Dataset Paper
3. ML-based IDS Research